

Assignment 3 Writeup

DO NOT TAG

Name: Umar Rafi
GT Email: urafi3@gatech.edu

Seq2Seq Results – Default configuration

RNN

Training Loss: 4.6141
Training Perplexity: 100.90
Validation Loss: 4.7015
Validation Perplexity: 110.11

LSTM

Training Loss: 3.1628
Training Perplexity: 23.64
Validation Loss: 3.3435
Validation Perplexity: 28.32

RNN-with-Attention

Training Loss: 3.3004
Training Perplexity: 27.12
Validation Loss: 3.4544
Validation Perplexity: 31.64

LSTM-with-Attention

Training Loss: 3.1205
Training Perplexity: 22.66
Validation Loss: 3.3054
Validation Perplexity: 27.26

Seq2Seq Explanation (RNN vs LSTM)

Compare your RNN result to your LSTM result and explain why they differ.

The LSTM significantly outperformed the plain RNN (validation perplexity 28.32 vs 110.11) due to fundamental differences in how each model handles long sequences.

A plain RNN updates its hidden state at every timestep by completely overwriting it: $h_t = \tanh(Wx_t + Wh_{t-1} + b)$. During backpropagation through many timesteps, gradients are multiplied repeatedly by the same weight matrix, causing them to vanish. In practice this means the RNN cannot retain information from early in the sentence by the time it reaches the end, making it unable to capture long-range dependencies between words.

The LSTM addresses this with a dedicated cell state and three gates. The forget gate decides what fraction of previous memory to keep. The input gate controls how much new information to write. The output gate controls what to expose as the hidden state. Critically, the cell state is updated additively ($c_t = f_t * c_{t-1} + i_t * g_t$) rather than being overwritten at each step — this creates a direct gradient pathway through time that largely prevents vanishing gradients and allows the model to carry relevant context across many timesteps.

For German-English translation this matters because German sentences frequently place verbs at the end. The RNN forgets the subject by the time it reaches the verb, while the LSTM stores that context in its cell state and retrieves it when needed, producing far more accurate translations and a much lower perplexity.

Seq2Seq Explanation (RNN vs RNN-with-Attention)

Compare your RNN result to your RNN-with-Attention result and explain why they differ.

Adding attention to the RNN dramatically improved performance, dropping validation perplexity from 110.11 to 31.64 — a reduction of over 70%.

A plain RNN encoder compresses the entire input sentence into a single fixed-size hidden vector that is passed to the decoder. This creates an information bottleneck — no matter how long the input sentence is, all meaning must be squeezed into one vector. By the time the decoder generates the last word of the translation, the information from the beginning of the source sentence has been largely lost.

Attention solves this bottleneck by giving the decoder access to all encoder hidden states at every decoding step, not just the final one. At each step the decoder computes a cosine similarity score between its current hidden state (the query) and every encoder output (the keys). These scores are passed through a softmax to produce attention weights that sum to 1. A weighted sum of the encoder outputs is then computed as the context vector, which is concatenated with the decoder input before being fed into the RNN.

This means at each translation step the decoder can selectively focus on the most relevant part of the source sentence. For example, when translating a verb, it can attend strongly to the corresponding verb in the source, rather than relying on a single compressed vector to contain all that information. This is especially important for German-English translation where word order differs significantly between the two languages.

The result is that RNN with attention (31.64) nearly matches plain LSTM (28.32), demonstrating that the information bottleneck was the primary weakness of the plain RNN, not the recurrent architecture itself.

Seq2Seq Results – Best model

Your best model after hyper-parameter tuning

Best model

Training Loss: 2.7746

Training Perplexity: 16.03

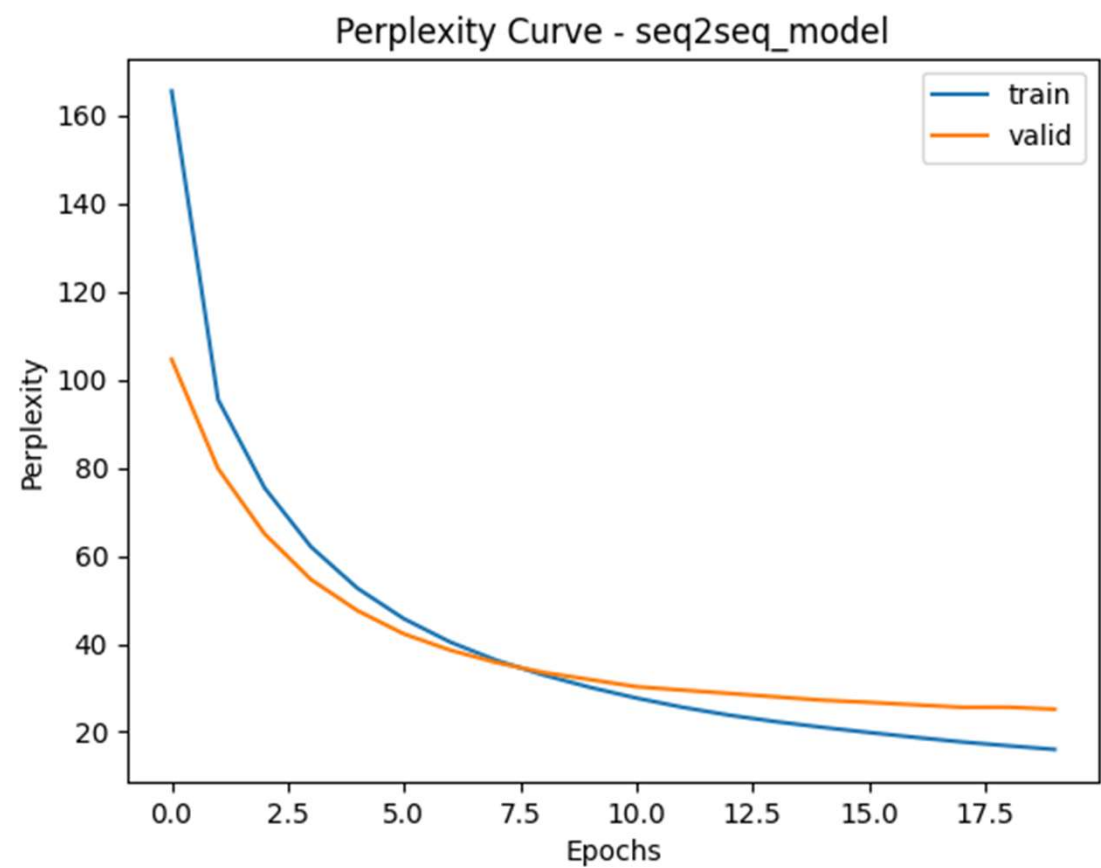
Validation Loss: 3.2255

Validation Perplexity: 25.17

List your best model hyper-parameter values including model type:

Best model is LSTM with Attention using embedding size 256, hidden size 256, dropout 0.1, learning rate 0.0005, trained for 20 epochs.

Seq2Seq Best model Learning Curves (Perplexity)



Seq2Seq Explanation – Best model

Explain the details of your best model. Explain what you did to improve your model's performance and why

The best Seq2Seq model used LSTM with attention, achieving a validation perplexity of 25.17 compared to 110.11 for the baseline plain RNN — an improvement of over 75%.

The primary improvements came from hyperparameter tuning: hidden and embedding sizes were increased from 128 to 256, dropout was reduced from 0.2 to 0.1, and the learning rate was lowered from 0.001 to 0.0005. Larger hidden states give the model more capacity to represent complex linguistic patterns, and the larger embedding allows each word to be represented by a richer vector. Reduced dropout suited the larger model since increased capacity already aided generalization, and the lower learning rate allowed more careful weight updates, preventing the optimizer from overshooting.

Attention was also added to the decoder. Without attention, the encoder must compress the entire source sentence into a single hidden vector, creating an information bottleneck. Attention allows the decoder to directly access all encoder hidden states at each step, computing a cosine similarity score to determine which parts of the source are most relevant. This is especially beneficial for German-English translation where word order differs significantly.

The LSTM architecture was used as the base, having already demonstrated stronger performance in the RNN vs LSTM comparison. Its gating mechanism and dedicated cell state allow it to retain long-range dependencies without vanishing gradients, which is critical when early context influences later word choices.

Together these changes reduced validation perplexity from 110.11 to 25.17, demonstrating that hyperparameter tuning, attention, and architecture choice all contribute meaningfully to translation quality.

Transformer Results – Encoder Only

Default configuration

Training Loss: 2.1525

Training Perplexity: 8.61

Validation Loss: 3.1178

Validation Perplexity: 22.60

Best model

Training Loss: 1.9272

Training Perplexity: 6.87

Validation Loss: 3.1812

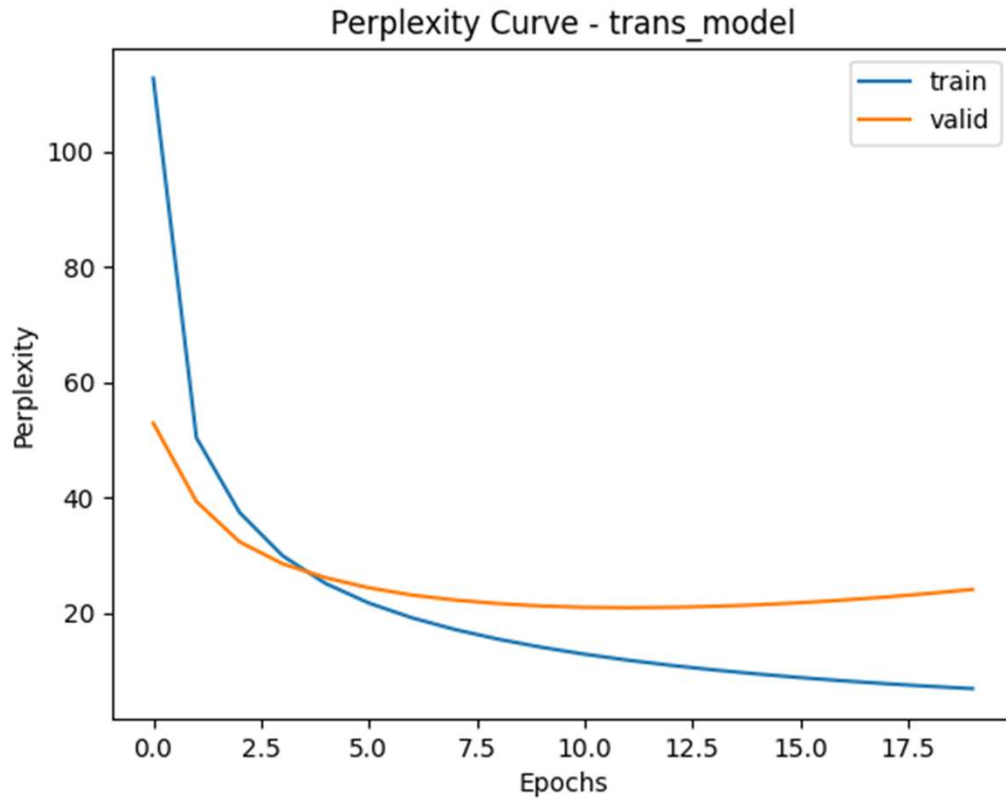
Validation Perplexity: 24.08

List your best model hyper-parameter values:

Since the default actually performed better, we will use these as our best model:

- Model type: Transformer Encoder Only
- Hidden dim: 128
- Number of heads: 2
- Feedforward dim: 2048
- Learning rate: 0.001
- Epochs: 10

Transformer (Encoder Only) Best model Learning Curves (Perplexity)



The training perplexity continues to decrease after epoch 10 while validation perplexity flattens and begins to rise slightly, indicating overfitting. This confirms that the default 10-epoch configuration produces the best validation perplexity of 22.60.

Transformer (Encoder Only) Explanation – Best model

Explain what you did here and why you did it to improve your model performance relative to default hyper parameters

The default Transformer encoder-only configuration used a learning rate of 0.001 and trained for 10 epochs, achieving a validation perplexity of 22.60. To attempt improvement, the learning rate was reduced to 0.0005 and epochs were increased to 20. The intuition was that a lower learning rate would allow more careful weight updates and more epochs would give the model more time to converge.

However, the tuned run produced a worse validation perplexity of 24.08 despite better training perplexity of 6.87 vs 8.61. The learning curve revealed the reason — after epoch 10 the training perplexity continued to decrease while validation perplexity flattened and began rising slightly, a classic sign of overfitting. The model was memorizing the training data rather than learning generalizable translation patterns.

This indicates the default 10-epoch configuration was already well-tuned for this dataset size. The Transformer has a large number of parameters relative to the training data, making it prone to overfitting with extended training. The default learning rate of 0.001 with 10 epochs provides the right balance between learning enough from the data and stopping before overfitting degrades generalization.

The default configuration therefore remains the best model with validation perplexity 22.60, which also outperforms the best Seq2Seq model (25.17), confirming that the Transformer's self-attention mechanism is more effective than RNN-based approaches for this translation task.

Transformer (Encoder Only) Translation Results, Best Model

Put translation results for your best model (1st 9 sentences) here

True Translation	Predicted Translation
<sos> a man in an orange hat starring at something <eos>	<sos> a man in an orange hat carving something <eos>
<sos> a boston terrier is running on lush green grass in front of a white fence <eos>	<sos> a boston staff is walking across the grass in front of a white fence <eos>
<sos> a girl in karate uniform breaking a stick with a front kick <eos>	<sos> a girl in a karate uniform is performing a board <eos>
<sos> five people wearing winter jackets and helmets stand in the snow with <unk> in the background <eos>	<sos> five people in winter clothes and helmets stand in the snow with <unk> in the background <eos>
<sos> people are fixing the roof of a house <eos>	<sos> people are fixing roof of a house <eos>
<sos> a group of people standing in front of an igloo <eos>	<sos> a group of people stand outside a mountain <eos>
<sos> a guy works on a building <eos>	<sos> a guy working on a building <eos>
<sos> a man in a vest is sitting in a chair and holding magazines <eos>	<sos> a man in a vest is sitting on a chair holding umbrellas <eos>
<sos> a mother and her young song enjoying a beautiful day outside <eos>	<sos> a mother and her son enjoying a beautiful day outdoors <eos>

Table 2

Seq2Seq (Best model) Translation Results

Put translation results for your best model (1st 9 sentences) here

True Translation	Predicted Translation
<sos> a man in an orange hat starring at something <eos>	<sos> a man in a hat hat is a <eos>
<sos> a boston terrier is running on lush green grass in front of a white fence <eos>	<sos> a brown brown dog is in in a field a a a white white <eos>
<sos> a girl in karate uniform breaking a stick with a front kick <eos>	<sos> a girl in a blue is is a a a a <eos>
<sos> five people wearing winter jackets and helmets stand in the snow with <unk> in the background <eos>	<sos> five people in blue and blue are in in the in background background background background background <eos>
<sos> people are fixing the roof of a house <eos>	<sos> people are the the of a a <eos>
<sos> a group of people standing in front of an igloo <eos>	<sos> a group of people standing in front of a <eos>
<sos> a guy works on a building <eos>	<sos> a guy working on a a <eos>
<sos> a man in a vest is sitting in a chair and holding magazines <eos>	<sos> a man in a black is sitting sitting a chair chair a <eos>
<sos> a mother and her young song enjoying a beautiful day outside <eos>	<sos> a mother and her daughter enjoying in in a a <eos>

Table 3

Compare Seq2Seq to Transformer Encoder Only (Best models)

Compare your Seq2Seq best model results to your Transformer best model results both quantitatively and qualitatively and explain the differences.

Quantitative Comparison:

The Transformer encoder-only model outperformed the best Seq2Seq model on validation perplexity — 22.60 vs 25.17. While both models were trained on the same dataset for the same number of epochs, the Transformer achieved lower perplexity with the default configuration while the Seq2Seq required larger hidden sizes, attention, and tuned hyperparameters to reach its best result.

Qualitative Comparison:

Looking at the translation outputs, the difference is even more striking. The Transformer produced fluent, coherent sentences that closely matched the true translations, getting the right words in the right order with only minor errors such as "carving" instead of "starring" or "umbrellas" instead of "magazines." The Seq2Seq model by contrast produced repetitive and incoherent output — words like "hat hat", "brown brown", "in in" and "background background background" appear repeatedly, and whole phrases are missing or garbled.

Why the Transformer performs better:

The Transformer's self-attention mechanism allows every word in the sequence to directly attend to every other word in a single layer, capturing global dependencies without having to pass information through many sequential timesteps. The Seq2Seq model processes words one at a time and must compress all source information through a single hidden state vector at each step, even with attention. For longer sentences this bottleneck causes the decoder to lose track of context and repeat itself.

Additionally, the Transformer processes all positions in parallel during training, which allows gradients to flow more effectively and the model to learn more complex patterns. The Seq2Seq model processes sequentially, making it harder to learn long-range relationships between distant words in the sentence.

Theory question

Problem 1: dL2/dU derivation

The RNN equations for each timestep are $a_t = Ux_t + Wh_{t-1} + b$, $h_t = \tanh(a_t)$, $o_t = Vh_t + c$, and $\hat{y}_t = \text{softmax}(o_t)$, with $L_t = \text{CE}(\hat{y}_t, y_t)$. U shows up in the computation of both a_1 and a_2 , so when I differentiate L_2 with respect to U I have to account for both places it enters — directly through a_2 , and indirectly through h_1 , since h_1 was computed using U back at $t=1$.

Starting from the loss and working backward, the combined softmax/cross-entropy gradient (given in the assignment) is:

$$\partial L_2 / \partial o_2 = \hat{y}_2 - y_2$$

I'll call this e_2 . Since $o_2 = Vh_2 + c$, this pushes back through V :

$$\partial L_2 / \partial h_2 = V^T e_2$$

and then through the tanh nonlinearity at a_2 , using $\tanh'(x) = 1 - \tanh^2(x)$:

$$\delta_2 = (V^T e_2) \odot (1 - h_2 \odot h_2)$$

This δ_2 is $\partial L_2 / \partial a_2$. Since $a_2 = Ux_2 + Wh_1 + b$ is linear in U , the direct piece of the gradient is just:

$$(\partial L_2 / \partial U)_{\text{direct}} = \delta_2 x_2^T$$

But a_2 also depends on h_1 through the Wh_1 term, and h_1 was itself built from U ($a_1 = Ux_1 + Wh_0 + b$). So I need to keep tracing back one more step. Continuing the chain rule through W :

$$\partial L_2 / \partial h_1 = W^T \delta_2$$

then through tanh again at a_1 :

$$\delta_1 = (W^T \delta_2) \odot (1 - h_1 \odot h_1)$$

which gives the second (indirect) contribution, since $a_1 = Ux_1 + Wh_0 + b$:

$$(\partial L_2 / \partial U)_{\text{indirect}} = \delta_1 x_1^T$$

Adding the two pieces together gives the full gradient:

$$\partial L_2 / \partial U = \delta_2 x_2^T + \delta_1 x_1^T$$

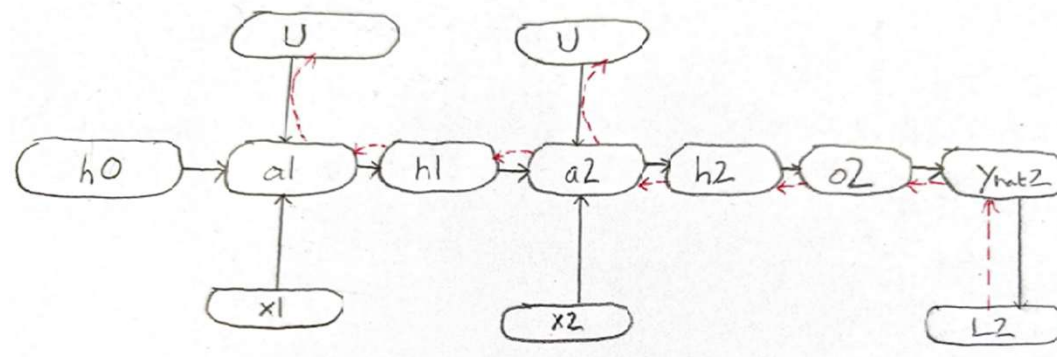
with

$$\delta_2 = (V^T(\hat{y}_2 - y_2)) \odot (1 - h_2 \odot h_2), \delta_1 = (W^T \delta_2) \odot (1 - h_1 \odot h_1)$$

The recursion stops at $t=1$ since h_0 is just the initial hidden state and isn't a function of U .

Computation graph

Below is the graph for L_2 as a function of U , showing the forward pass (solid) and the two backward paths that both terminate at U (dashed, red) — one direct through x_2 , one indirect through h_1 and x_1 .



solid = forward pass

dashed red = backward gradient path for $\partial L_2 / \partial U$

Paper discussion

Do Vision Transformers See Like Convolutional Neural Networks?

Review

This paper investigates whether Vision Transformers (ViT) and CNNs, despite achieving similar accuracies, actually solve vision tasks the same way internally. Using CKA, linear probes, and receptive field analysis, the authors uncover systematic structural differences. ViTs have much more uniform layer representations due to unusually strong skip connections that propagate the CLS token and spatial features largely unchanged through the network. I appreciated the addition of the residual block norm analysis—it really clarifies why the representations stay so uniform: the identity path simply dominates the residual path in ViT, which is quite different from ResNet where both paths contribute more evenly. Self-attention gives ViTs early global access, but this flexibility only pays off with large-scale pretraining; without enough data, they fail to learn meaningful local attention patterns and underperform. They also find ViTs preserve spatial location far better into their final layers, suggesting they are better suited for dense prediction tasks like object detection.

Methodologically, I liked that they use multiple complementary tools rather than relying on a single metric—that makes the results much more convincing. The intervention experiments (removing skips, varying data scale, comparing CLS vs. GAP pooling) go beyond just correlational observation and actually test causal claims directly, which I thought was a strong point. The finding that removing skip connections from ViT causes its representations to become more CNN-like is particularly compelling, as it isolates the mechanism rather than just observing the effect.

That said, I'm a bit skeptical of their heavy reliance on CKA. Compressing high-dimensional representations into a single similarity scalar risks oversimplifying things, which they do acknowledge in their limitations. Also, since most of the analysis sticks to JFT/ImageNet classification, I'm left wondering how well these findings transfer to smaller-scale domains or dense tasks like segmentation—they don't really test that empirically. The conclusion that data scale is the primary lever for ViT performance is well-supported within their experimental setup, but broader validation across domains and task types would strengthen the claim considerably.

Paper discussion

Question 1: Compare and contrast the learned features of ViTs and CNNs

Architecturally, the difference is stark: CNNs hardcode local, spatially-equivariant receptive fields—early layers only see nearby pixels, building up context gradually through depth. This hardcoded locality gives CNNs translation equivariance for free, whereas ViTs need to learn that property through data. ViTs replace this with self-attention, so every patch can attend globally from the very first layer.

Empirically, the CKA analysis shows CNNs have distinct staged representations—early layers look very different from deeper ones—while ViTs show a surprisingly uniform similarity pattern across almost all layers. What struck me is that the cross-model comparisons quantify this starkly: it takes roughly 5-6 layers of a ResNet to match the representational content found in just ViT's first 2 layers. This means ViTs aren't just skipping shallow feature extraction; they're effectively compressing what CNNs do over several layers into a single self-attention operation.

I think this comes down to two or three things. First, self-attention gives ViTs early access to global information; even in the lowest layers, attention heads split into local and global groups, whereas CNNs are architecturally forced to be local early on. Second, ViT's skip connections are much stronger than ResNet's—they pass the CLS token through mostly unchanged, which keeps representations looking similar across depth. Third, the attention heads in early ViT layers diversify quickly; some focus purely on local high-frequency edges (mimicking CNN conv1), while others look at global shape, something CNNs can't do until much later.

However, it's not that ViTs ignore local features entirely. Give them enough pretraining data (JFT), and their early heads learn local filters anyway, mimicking a CNN's inductive bias. But this behavior is learned rather than hardcoded, and it disappears when trained on smaller datasets like ImageNet alone. To me, this highlights a fundamental trade-off. If you have the compute and data, ViTs are incredibly flexible. But if you don't, CNN's hardcoded bias is a massive efficiency boost. So is ViT "better"? Not universally—it depends entirely on your budget and data scale.

Paper discussion

Question 2: What is spatial localization, and why might ViTs be better suited for object detection?

Spatial localization measures how faithfully a model's internal feature map preserves the coordinate mapping of the input image. If localization is strong, a feature at grid position (i,j) primarily encodes information from input patch (i,j) , even after passing through many non-linear transformations. If it's weak, that feature vector becomes a mixed-up average of surrounding patches, effectively blurring the positional information. A model with good localization retains a tight, one-to-one correspondence between where a feature came from and what it represents, even deep into the network.

The paper finds ViT tokens (when trained with a CLS token) remain remarkably localized—even at the final layer, a token's representation correlates most strongly with its original patch. Interestingly, they note slight "bleeding" only between adjacent edge patches in ViT, which makes intuitive sense since those patches often contain continuous objects or boundaries. ResNet, in contrast, shows much weaker localization; its final-layer features correlate broadly across multiple spatial regions rather than concentrating on the original one, indicating the positional signal is thoroughly smeared out.

I think the key explanation here is the training objective. ResNet uses Global Average Pooling for classification, which sums over the entire spatial dimension and actively encourages the network to discard spatial detail since only the aggregate signal matters for the final prediction. GAP destroys the ordered structure necessary for precise coordinate regression. In contrast, ViT's separate CLS token absorbs the global signal, leaving the spatial (non-CLS) tokens free to preserve their exact locations. The authors confirm this neatly with an ablation: when ViT is retrained using GAP instead of CLS, its localization drops substantially, behaving more like a CNN.

Why does this matter for object detection? Detection fundamentally requires regression—predicting exact (x, y, w, h) bounding box coordinates. If your backbone produces smeared features, the detection head receives noisy gradients when trying to refine box edges. Modern detectors use Feature Pyramid Networks that rely on spatially sharp feature maps at multiple scales. A ViT backbone, which preserves crisp spatial localization deep into the network, supplies cleaner semantic maps for the detection head, improving box prediction accuracy especially at object boundaries.